# Unidade didáctica 3. Consultas SQL

Nesta unidade aprenderemos a realizar consultas sobre bases de datos relacionais mediante SQL. Traballaremos coa sentenza `SELECT` para recuperar información, seleccionar columnas, filtrar rexistros e ordenar os resultados.

Os exemplos realizaranse utilizando MariaDB/MySQL e MySQL Workbench. Tamén se explicará como executar as consultas directamente desde a consola do sistema.

## 1. Introdución ás consultas SQL

### 1.1. Que é unha consulta SQL?

Unha consulta SQL é unha instrución que permite recuperar información almacenada nunha base de datos. As consultas utilízanse para obter datos dunha ou varias táboas sen modificar o contido almacenado.

A sentenza principal para consultar información é `SELECT`. A súa finalidade é indicar que datos queremos recuperar e de que táboa ou táboas proceden.

> **Importante:** As consultas realizadas con `SELECT` non modifican os datos da base de datos. Úsanse exclusivamente para recuperar e visualizar información.

### 1.2. Ferramentas para executar consultas

Existen diferentes ferramentas que permiten escribir e executar instrucións SQL. Durante esta unidade utilizaremos principalmente MySQL Workbench e a consola de MariaDB/MySQL.

#### MySQL Workbench

MySQL Workbench é unha ferramenta gráfica que permite administrar bases de datos e executar sentenzas SQL.

Para executar unha consulta:

1. Iniciar MySQL Workbench.
2. Conectarse ao servidor de bases de datos.
3. Seleccionar a base de datos que se vai utilizar.
4. Escribir a consulta no editor SQL.
5. Executar a instrución mediante o botón de execución ou a combinación Ctrl + Enter.

6. Consultar os resultados na zona inferior do editor.

### Consola de MariaDB/MySQL

Tamén podemos executar consultas directamente desde o terminal. Para iso debemos iniciar sesión no servidor de bases de datos.

```
mysql -u root -p
```

Introducimos o contrasinal correspondente. Unha vez dentro do cliente, podemos executar instrucións SQL directamente.

```
SHOW DATABASES;
```

Para seleccionar unha base de datos utilizamos `USE`.

```
USE centro_educativo;
```

A partir dese momento, as consultas executaranse sobre a base de datos seleccionada, salvo que indiquemos explicitamente outra.

## 1.3. Estrutura básica de SELECT

A estrutura máis sinxela dunha consulta SQL é a seguinte:

```
SELECT columnas
FROM nome_taboa;
```

As dúas partes principais son:

- **SELECT:** indica as columnas que queremos recuperar.
- **FROM:** indica a táboa da que proceden os datos.

### Exemplo 1. Consultar todos os alumnos

```
SELECT *
FROM alumnos;
```

O símbolo `*` indica que queremos recuperar todas as columnas da táboa.

#### Exemplo 2. Consultar todos os cursos

```
SELECT *
FROM cursos;
```

### 1.4. Terminación das instrucións

As instrucións SQL adoitan finalizar cun punto e coma (`;`). Este carácter indica o final da sentenza.

```
SELECT * FROM alumnos;

SELECT * FROM cursos;
```

SQL non distingue normalmente entre maiúsculas e minúsculas nas palabras reservadas. Por convención, escribiremos as palabras clave en maiúsculas para facilitar a lectura.

> **Boas prácticas:** Recoméndase escribir cada cláusula da consulta nunha liña diferente e utilizar maiúsculas para as palabras reservadas.

## 2. Proxección de rexistros

### 2.1. Selección de columnas

A proxección consiste en seleccionar que columnas dunha táboa queremos mostrar nunha consulta.

Non sempre necesitamos recuperar toda a información dunha táboa. Podemos seleccionar unicamente os campos que nos interesan.

```
SELECT columna1, columna2
FROM nome_taboa;
```

#### Exemplo 1. Consultar os nomes dos alumnos

```
SELECT nome
FROM alumnos;
```

O resultado mostrará unicamente a columna `nome`.

### Exemplo 2. Consultar o nome e a idade

```
SELECT nome, idade
FROM alumnos;
```

A consulta recupera dúas columnas da táboa `alumnos`.

### Exemplo 3. Consultar o nome e as horas dos cursos

```
SELECT nome, horas
FROM cursos;
```

## 2.2. Uso do asterisco (*)

Cando queremos recuperar todas as columnas dunha táboa podemos utilizar o carácter `*`.

```
SELECT *
FROM alumnos;
```

Esta consulta devolve todas as columnas e todos os rexistros da táboa.

> **Recomendación:** Aínda que o uso de `*` é cómodo, nas consultas habituais é preferible indicar explicitamente as columnas necesarias. Isto facilita a lectura das consultas e evita recuperar información innecesaria.

## 2.3. Eliminación de rexistros duplicados con DISTINCT

Por defecto, SQL devolve todos os rexistros que cumpren as condicións da consulta, incluíndo os valores repetidos.

A palabra reservada `DISTINCT` permite eliminar os resultados duplicados.

```
SELECT DISTINCT columna
FROM nome_taboa;
```

#### Exemplo 1. Consultar as idades diferentes dos alumnos

```
SELECT DISTINCT idade
FROM alumnos;
```

Se varios alumnos teñen a mesma idade, esa idade aparecerá unha única vez no resultado.

#### Exemplo 2. Consultar os cursos diferentes

```
SELECT DISTINCT curso_id
FROM alumnos;
```

A consulta mostra os identificadores dos cursos nos que hai alumnado, sen repetir ningún valor.

### 2.4. DISTINCT sobre varias columnas

Podemos aplicar `DISTINCT` sobre varias columnas. Neste caso elimínanse as combinacións de valores duplicadas.

```
SELECT DISTINCT idade, curso_id
FROM alumnos;
```

Dúas filas só se consideran duplicadas cando coinciden todos os valores das columnas seleccionadas.

> **Para lembrar:**
>
> - `SELECT *` : recupera todas as columnas.
> - `SELECT nome, idade` : recupera columnas concretas.
> - `SELECT DISTINCT idade` : elimina valores repetidos.

## 3. Filtrado de rexistros con WHERE

### 3.1. Que é o filtrado de rexistros?

Ata agora realizamos consultas que devolvían todos os rexistros dunha táboa. Con frecuencia necesitamos recuperar só aqueles que cumpren determinadas condicións.

A cláusula `WHERE` permite establecer condicións para seleccionar os rexistros que queremos visualizar.

```
SELECT columnas
FROM nome_taboa
WHERE condicion;
```

A condición avalíase para cada rexistro. Só se mostran aqueles nos que o resultado da condición é verdadeiro.

#### Exemplo 1. Consultar alumnos maiores de idade

```
SELECT *
FROM alumnos
WHERE idade >= 18;
```

Só aparecen os alumnos cuxa idade é maior ou igual a 18.

#### Exemplo 2. Consultar os cursos con máis de 150 horas

```
SELECT *
FROM cursos
WHERE horas > 150;
```

### 3.2. Operadores de comparación

Os operadores de comparación permiten establecer condicións comparando o valor dunha columna cun valor determinado.

| Operador | Significado | Exemplo |
|---|---|---|
| `=` | Igual | `idade = 18` |
| `<>` | Distinto | `idade <> 18` |
| `!=` | Distinto | `idade != 18` |
| `>` | Maior que | `idade > 18` |
| `<` | Menor que | `idade < 18` |
| `>=` | Maior ou igual que | `idade >= 18` |
| `<=` | Menor ou igual que | `idade <= 18` |

### 3.3. Comparación de valores numéricos

Cando traballamos con columnas numéricas podemos utilizar os operadores matemáticos de comparación.

#### Exemplo 1. Alumnos cunha idade concreta

```
SELECT *
FROM alumnos
WHERE idade = 20;
```

#### Exemplo 2. Cursos con 200 horas

```
SELECT *
FROM cursos
WHERE horas = 200;
```

### 3.4. Comparación de cadeas de texto

Para comparar valores de texto utilizamos normalmente o operador `=`. Os valores de tipo texto deben escribirse entre comiñas simples.

```
SELECT *
FROM alumnos
WHERE nome = 'Ana García';
```

Tamén podemos buscar valores diferentes:

```
SELECT *
FROM alumnos
WHERE nome <> 'Ana García';
```

> **Atención:** Os valores de texto deben escribirse entre comiñas simples. Non debemos confundir os nomes das columnas cos valores que queremos comparar.

# 4. Operadores lóxicos

## 4.1. Uso de AND

O operador `AND` permite combinar varias condicións. Para que un rexistro apareza no resultado, todas as condicións deben cumprirse.

```
SELECT columnas
FROM nome_taboa
WHERE condicion1 AND condicion2;
```

### Exemplo. Alumnos maiores de idade que teñen 20 anos ou máis

```
SELECT *
FROM alumnos
WHERE idade >= 18
AND idade <= 25;
```

Só se mostrarán os alumnos que teñan unha idade comprendida entre 18 e 25 anos, ambos incluídos.

## 4.2. Uso de OR

O operador `OR` permite combinar condicións nas que abonda con que se cumpra unha delas.

```
SELECT columnas
FROM nome_taboa
WHERE condicion1 OR condicion2;
```

### Exemplo. Consultar alumnos de 18 ou 20 anos

```
SELECT *
FROM alumnos
WHERE idade = 18
OR idade = 20;
```

Aparecerán os alumnos que teñan 18 anos ou 20 anos.

## 4.3. Uso de NOT

O operador `NOT` nega unha condición. Selecciona os rexistros que non cumpren a condición indicada.

### Exemplo. Alumnos que non teñen 20 anos

```
SELECT *
FROM alumnos
WHERE NOT idade = 20;
```

Esta consulta equivale a utilizar o operador `<>`:

```
SELECT *
FROM alumnos
WHERE idade <> 20;
```

## 4.4. Combinación de operadores lóxicos

Podemos combinar `AND`, `OR` e `NOT` nunha mesma consulta para establecer condicións máis complexas.

Cando combinamos diferentes operadores é recomendable utilizar parénteses para indicar claramente a orde na que deben avaliarse as condicións.

### Exemplo. Alumnos de 18 anos ou alumnos de 20 anos que pertencen ao curso 1

```
SELECT *
FROM alumnos
WHERE idade = 18
OR (idade = 20 AND curso_id = 1);
```

A consulta selecciona:

- Todos os alumnos que teñen 18 anos.
- Os alumnos que teñen 20 anos e pertencen ao curso 1.

### 4.5. Prioridade dos operadores

SQL establece unha orde de prioridade para avaliar os operadores lóxicos:

1. `NOT`
2. `AND`
3. `OR`

Isto significa que `AND` se avalía antes que `OR`. Se queremos cambiar esta orde debemos utilizar parénteses.

> **Importante:** Non debemos confiar na prioridade dos operadores cando unha consulta combina varias condicións. O uso de parénteses fai que a consulta sexa máis clara e evita resultados inesperados.

## 5. Outros operadores de filtrado

### 5.1. Operador BETWEEN

O operador `BETWEEN` permite comprobar se un valor está dentro dun intervalo.

```
SELECT columnas
FROM nome_taboa
WHERE columna BETWEEN valor1 AND valor2;
```

Os valores inicial e final están incluídos no intervalo.

### Exemplo. Alumnos con idades entre 18 e 22 anos

```
SELECT *
FROM alumnos
WHERE idade BETWEEN 18 AND 22;
```

Esta consulta equivale a:

```
SELECT *
FROM alumnos
WHERE idade >= 18
AND idade <= 22;
```

## 5.2. Operador IN

O operador `IN` permite comprobar se o valor dunha columna coincide con algún dos valores dunha lista.

```
SELECT columnas
FROM nome_taboa
WHERE columna IN (valor1, valor2, valor3);
```

### Exemplo. Consultar alumnos dos cursos 1 e 3

```
SELECT *
FROM alumnos
WHERE curso_id IN (1, 3);
```

Esta consulta equivale a utilizar varias condicións unidas mediante `OR`:

```
SELECT *
FROM alumnos
WHERE curso_id = 1
OR curso_id = 3;
```

Tamén podemos utilizar `NOT IN` para seleccionar os rexistros cuxo valor non aparece na lista.

```
SELECT *
FROM alumnos
WHERE curso_id NOT IN (1, 3);
```

## 5.3. Operador LIKE

O operador `LIKE` permite buscar patróns dentro dunha cadea de texto. Utilízase principalmente cando non coñecemos o valor completo que queremos buscar.

Para construír os patróns utilízanse dous caracteres especiais:

| Carácter | Significado | Exemplo |
|---|---|---|
| `%` | Representa cero ou máis caracteres. | `'A%'` |
| `_` | Representa exactamente un carácter. | `'A_a'` |

### Exemplo 1. Nomes que comezan pola letra A

```
SELECT *
FROM alumnos
WHERE nome LIKE 'A%';
```

O símbolo `%` indica que despois da letra A pode aparecer calquera secuencia de caracteres.

### Exemplo 2. Nomes que terminan en z

```
SELECT *
FROM alumnos
WHERE nome LIKE '%z';
```

### Exemplo 3. Nomes que conteñen a palabra García

```
SELECT *
FROM alumnos
WHERE nome LIKE '%García%';
```

### Exemplo 4. Nomes que comezan por A e teñen tres caracteres

```
SELECT *
FROM alumnos
WHERE nome LIKE 'A__';
```

Cada guión baixo representa exactamente un carácter.

> **Para lembrar:**
>
> - `'A%'` : comeza por A.
> - `'%A'` : termina en A.
> - `'%A%'` : contén a letra A.
> - `'A_'` : comeza por A e ten dous caracteres.

## 5.4. Operador IS NULL

Nas bases de datos podemos atopar columnas que non teñen ningún valor asignado. Estes casos represéntanse mediante o valor especial `NULL`.

`NULL` non é o mesmo que cero, unha cadea baleira ou un espazo en branco. Representa a ausencia dun valor.

Para comprobar se unha columna contén un valor nulo utilizamos `IS NULL`.

### Exemplo 1. Alumnos sen curso asignado

```
SELECT *
FROM alumnos
WHERE curso_id IS NULL;
```

Para recuperar os rexistros nos que a columna si ten un valor utilizamos `IS NOT NULL`.

### Exemplo 2. Alumnos que teñen un curso asignado

```
SELECT *
FROM alumnos
WHERE curso_id IS NOT NULL;
```

> **Erro habitual:** Non debemos utilizar `= NULL` nin `<> NULL`. As comparacións con NULL deben realizarse mediante `IS NULL` e `IS NOT NULL`.

### 5.5. Combinación de operadores

Todos os operadores anteriores poden combinarse mediante `AND`, `OR` e `NOT`.

#### Exemplo. Alumnos maiores de idade dos cursos 1 ou 2

```
SELECT *
FROM alumnos
WHERE idade >= 18
AND curso_id IN (1, 2);
```

#### Exemplo. Alumnos cuxo nome comeza por A ou por M

```
SELECT *
FROM alumnos
WHERE nome LIKE 'A%'
OR nome LIKE 'M%';
```

#### Exemplo. Cursos con máis de 100 e menos de 200 horas

```
SELECT *
FROM cursos
WHERE horas > 100
AND horas < 200;
```

## 6. Ordenación dos resultados con ORDER BY

### 6.1. Que é a ordenación?

Por defecto, unha consulta SQL non garante unha orde concreta nos rexistros devoltos. Se necesitamos presentar os resultados nunha orde determinada debemos utilizar a cláusula `ORDER BY`.

```
SELECT columnas
FROM nome_taboa
ORDER BY columna;
```

Podemos ordenar os resultados utilizando columnas numéricas, cadeas de texto ou datas.

## 6.2. Ordenación ascendente

A palabra reservada `ASC` indica que os resultados deben ordenarse de forma ascendente.

- Os números ordénanse de menor a maior.
- O texto ordénase alfabeticamente.
- As datas ordénanse da máis antiga á máis recente.

### Exemplo. Ordenar os alumnos polo nome

```
SELECT *
FROM alumnos
ORDER BY nome ASC;
```

O modificador `ASC` é opcional, xa que é a ordenación predeterminada.

```
SELECT *
FROM alumnos
ORDER BY nome;
```

## 6.3. Ordenación descendente

A palabra reservada `DESC` permite ordenar os resultados de forma descendente.

- Os números ordénanse de maior a menor.
- O texto ordénase en orde alfabética inversa.
- As datas ordénanse da máis recente á máis antiga.

### Exemplo. Ordenar os cursos polo número de horas

```
SELECT *
FROM cursos
ORDER BY horas DESC;
```

Os cursos con maior número de horas aparecerán primeiro.

## 6.4. Ordenación por varias columnas

Podemos ordenar os resultados utilizando máis dunha columna. Neste caso, SQL aplica a ordenación seguindo a orde na que aparecen as columnas dentro de `ORDER BY`.

```
SELECT *
FROM alumnos
ORDER BY curso_id ASC, nome ASC;
```

A consulta ordena primeiro polo identificador do curso. Cando varios alumnos pertencen ao mesmo curso, ordénaos alfabeticamente polo nome.

### Exemplo. Ordenar por curso e idade

```
SELECT *
FROM alumnos
ORDER BY curso_id ASC, idade DESC;
```

Os alumnos aparecen agrupados por curso e, dentro de cada curso, ordenados de maior a menor idade.

## 6.5. Combinación de WHERE e ORDER BY

Podemos combinar o filtrado de rexistros coa ordenación dos resultados nunha mesma consulta.

### Exemplo. Consultar os alumnos maiores de idade ordenados por idade

```
SELECT *
FROM alumnos
WHERE idade >= 18
ORDER BY idade DESC;
```

Primeiro selecciónanse os alumnos que cumpren a condición e despois ordénanse os resultados.

### 6.6. Orde das cláusulas nunha consulta

Nas consultas que estamos traballando, a orde das cláusulas debe ser a seguinte:

```
SELECT columnas
FROM nome_taboa
WHERE condicion
ORDER BY columna ASC;
```

Non podemos colocar `ORDER BY` antes de `WHERE`. A estrutura da consulta debe respectar esta orde.

> **Resumo:**
>
> - `WHERE` selecciona os rexistros que cumpren unha condición.
> - `ORDER BY` ordena os rexistros devoltos.
> - `ASC` establece unha orde ascendente.
> - `DESC` establece unha orde descendente.
> - Pódense ordenar os resultados por varias columnas.

## 7. Resumo das principais sentenzas e operadores

| Elemento | Función | Exemplo |
|---|---|---|
| `SELECT` | Seleccionar columnas. | `SELECT nome` |
| `*` | Seleccionar todas as columnas. | `SELECT *` |
| `DISTINCT` | Eliminar resultados duplicados. | `SELECT DISTINCT idade` |
| `FROM` | Indicar a táboa. | `FROM alumnos` |
| `WHERE` | Filtrar rexistros. | `WHERE idade >= 18` |
| `AND` | Todas as condicións deben cumprirse. | `idade >= 18 AND idade <= 25` |
| `OR` | Abonda con que se cumpra unha condición. | `idade = 18 OR idade = 20` |
| `NOT` | Negar unha condición. | `NOT idade = 20` |
| `BETWEEN` | Comprobar un intervalo. | `idade BETWEEN 18 AND 25` |
| `IN` | Comprobar se un valor está nunha lista. | `curso_id IN (1, 2)` |
| `LIKE` | Buscar patróns de texto. | `nome LIKE 'A%'` |
| `IS NULL` | Buscar valores nulos. | `curso_id IS NULL` |
| `IS NOT NULL` | Buscar valores non nulos. | `curso_id IS NOT NULL` |
| `ORDER BY` | Ordenar os resultados. | `ORDER BY nome` |
| `ASC` | Ordenación ascendente. | `ORDER BY idade ASC` |
| `DESC` | Ordenación descendente. | `ORDER BY idade DESC` |

## 8. Exercicios de consolidación

Os seguintes exercicios permiten practicar os conceptos explicados ao longo destas seis sesións. As consultas deben executarse sobre a base de datos utilizada durante a unidade.

## Bloque 1. Consultas básicas

1. Mostrar todos os rexistros da táboa `alumnos`.
2. Mostrar todos os rexistros da táboa `cursos`.
3. Mostrar unicamente os nomes dos alumnos.
4. Mostrar o nome e a idade de todos os alumnos.
5. Mostrar o nome e o número de horas de cada curso.
6. Mostrar as idades diferentes que aparecen na táboa de alumnos.
7. Mostrar os identificadores dos cursos diferentes nos que hai alumnos.

## Bloque 2. Filtrado de rexistros

8. Mostrar os alumnos que teñen máis de 18 anos.
9. Mostrar os alumnos que teñen exactamente 20 anos.
10. Mostrar os cursos que teñen máis de 150 horas.
11. Mostrar os cursos que teñen menos de 200 horas.
12. Mostrar os alumnos que teñen entre 18 e 22 anos.
13. Mostrar os alumnos que pertencen aos cursos 1 e 2.
14. Mostrar os alumnos que non pertencen ao curso 1.

## Bloque 3. Operadores lóxicos e patróns

15. Mostrar os alumnos maiores de idade que pertencen ao curso 1.
16. Mostrar os alumnos que teñen 18 ou 20 anos.
17. Mostrar os alumnos que pertencen ao curso 1 ou ao curso 3 e teñen máis de 18 anos.
18. Mostrar os alumnos cuxo nome comeza pola letra A.
19. Mostrar os alumnos cuxo nome contén a letra a.
20. Mostrar os alumnos cuxo nome termina en z.
21. Mostrar os alumnos que non teñen curso asignado.
22. Mostrar os alumnos que teñen un curso asignado.

## Bloque 4. Ordenación

23. Mostrar todos os alumnos ordenados alfabeticamente polo nome.
24. Mostrar os alumnos ordenados de maior a menor idade.
25. Mostrar os cursos ordenados polo número de horas de menor a maior.

26. Mostrar os cursos ordenados polo número de horas de maior a menor.
27. Mostrar os alumnos ordenados primeiro polo identificador do curso e despois polo nome.
28. Mostrar os alumnos maiores de idade ordenados de maior a menor idade.

> **Recomendación para resolver os exercicios:** Antes de escribir unha consulta, identifica que columnas necesitas, de que táboa proceden os datos, que condicións deben cumprir os rexistros e se é necesario ordenar os resultados.

## 9. Estrutura xeral dunha consulta SQL

Despois de estudar os conceptos anteriores, podemos combinar as distintas cláusulas nunha mesma consulta.

```
SELECT DISTINCT columna1, columna2
FROM nome_taboa
WHERE condicion1
  AND condicion2
ORDER BY columna1 ASC;
```

Nesta estrutura:

1. `SELECT` indica as columnas que queremos recuperar.
2. `DISTINCT` elimina as combinacións duplicadas.
3. `FROM` indica a táboa de orixe.
4. `WHERE` establece as condicións de selección.
5. `ORDER BY` ordena os resultados.

### Exemplo final. Consulta combinada

Queremos obter os nomes e as idades dos alumnos maiores de idade que pertencen aos cursos 1 ou 2. Os resultados deben aparecer ordenados por idade de maior a menor.

```
SELECT nome, idade
FROM alumnos
WHERE idade >= 18
AND curso_id IN (1, 2)
ORDER BY idade DESC;
```

Esta consulta combina a selección de columnas, o filtrado mediante operadores de comparación e a ordenación dos resultados.